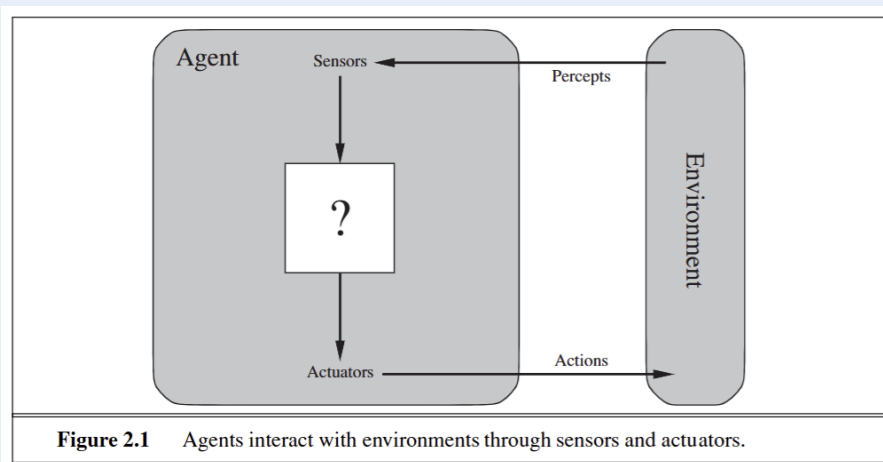


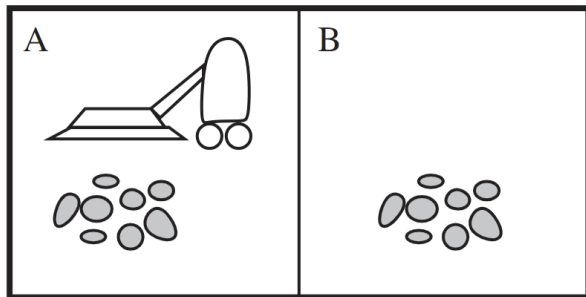
Intelligent Agents and Search Algorithms

Agents

- An agent is anything that perceives its environment through sensors and acts via actuators
- Percept sequence: complete history of everything the agent has perceived
- Agent function: maps any percept sequence to an action $f: P^* \rightarrow A$



Example



A vacuum-cleaner world with just two locations.

Percept sequence	Action
$[A, \textit{Clean}]$	<i>Right</i>
$[A, \textit{Dirty}]$	<i>Suck</i>
$[B, \textit{Clean}]$	<i>Left</i>
$[B, \textit{Dirty}]$	<i>Suck</i>
$[A, \textit{Clean}], [A, \textit{Clean}]$	<i>Right</i>
$[A, \textit{Clean}], [A, \textit{Dirty}]$	<i>Suck</i>
$\vdots$	$\vdots$
$[A, \textit{Clean}], [A, \textit{Clean}], [A, \textit{Clean}]$	<i>Right</i>
$[A, \textit{Clean}], [A, \textit{Clean}], [A, \textit{Dirty}]$	<i>Suck</i>
$\vdots$	$\vdots$

Figure 2.3 Partial tabulation of a simple agent function for the vacuum-cleaner world shown in Figure 2.2.

Specifying Agents: PEAS

To design an agent, specify its task environment with PEAS:

§1 Agents

PEAS	Component	Taxi Driver Example	Medical Diagnosis Example
P	Performance Measure	Safe trip, fast, legal, comfortable, profit	Healthy patient, minimise cost & risk
E	Environment	Roads, traffic, pedestrians, weather	Patient, hospital, staff, records
A	Actuators	Steering, accelerator, brake, horn, display	Test orders, treatments, referrals
S	Sensors	Cameras, speedometer, GPS, microphone	Symptoms, vitals, lab results

Rational Agents

For each possible percept sequence, a rational agent should select an action that is expected to maximize its performance measure, given the evidence provided by the percept sequence and whatever built-in knowledge the agent has.

Types of Environments

Observable

Fully observable / Partially observable

Can the agent see the full state?

Deterministic

Deterministic / Stochastic

Do actions have certain outcomes?

Episodic

Episodic / Sequential

Do past actions affect the future?

Dynamic

Static / Dynamic / Semidynamic

Does the world change while deciding?

Discrete/Cont.

Discrete / Continuous

Finite or infinite percepts/actions?

Agents

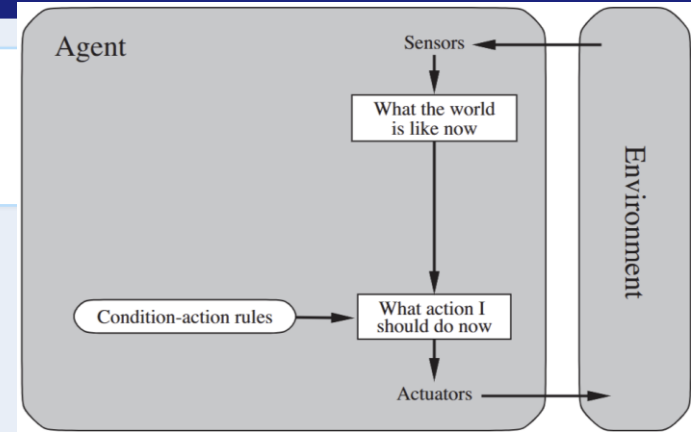
**Single-agent / Multi-agent
(comp/coop)**

Are other agents in the environment?

Basic Agent Architectures

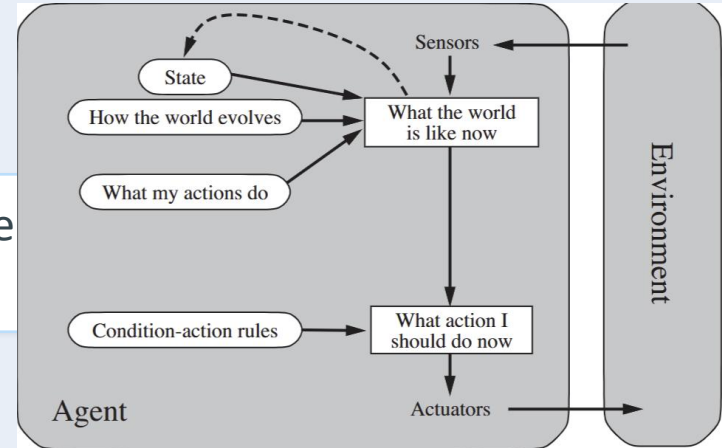
Simple Reflex

Maps current percept $\rightarrow$ action
If dirt, suck. Ignores history.



Model-Based Reflex

Maintains internal world state
Handles partial observability



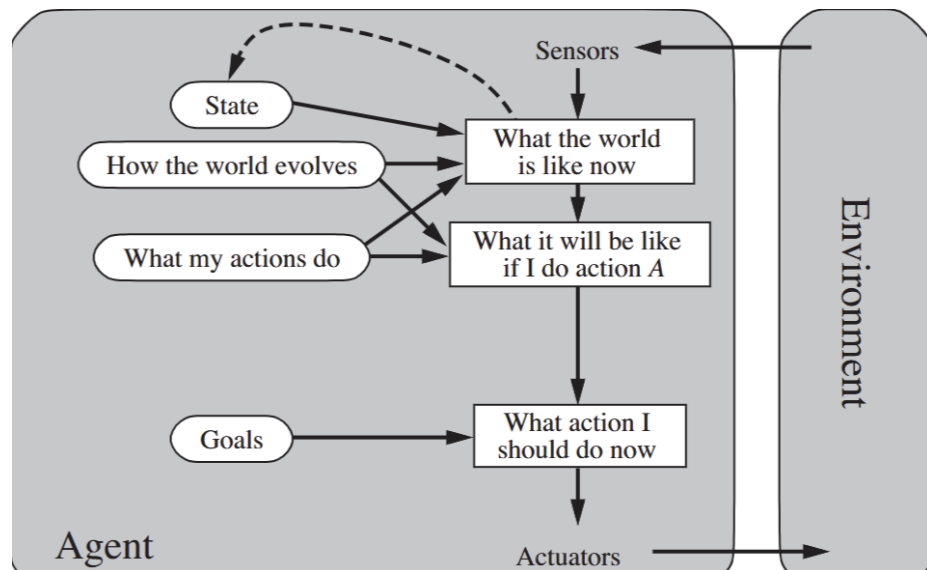
Basic Agent Architectures

Goal-Based

Considers future states & goals
Search and planning required

Two types of goal-based agents based on how they represent the world

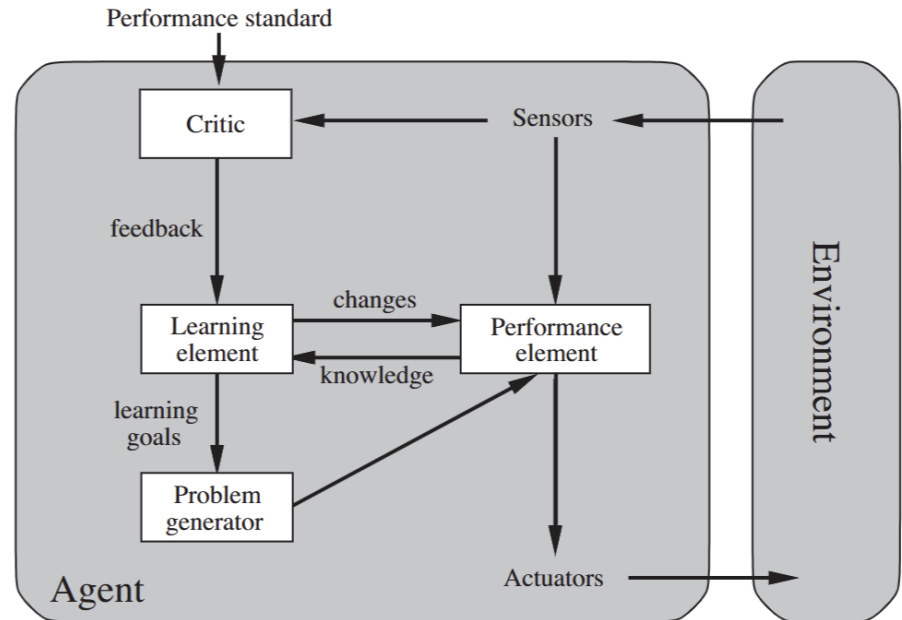
- Atomic representations (problem-solving agents) [Covered today]
- Factored representations (planning agents)



Basic Agent Architectures

Learning Agents

Improves through experience
Performance + critic + learning



Problem-solving agents

- Intelligent agents are supposed to maximize their performance measure.
- Performance is measured in terms of satisfaction of a given goal.
- Environment is represented as a state, there may be many environments or states.
- Goal is identification of a subset of the states
- Agent is assumed to be able to observe the “current” environment. Assume environment is discrete.
- Agent can take a set of discrete actions
- Agent knows deterministically how the environment will change as a result of an action.
- Problem-solving: starting from an initial state, find the sequence of actions that lead to one of the goal states.
- The process of looking for a sequence of actions that reaches the goal is called search.

Turning a task into a Search Problem

§2 Formulation

- A search problem has five components:
 - State space S — set of all possible world configurations
 - Initial state s_0 — where the agent starts
 - Actions $A(s)$ — choices available in state s
 - Transition model $\text{Result}(s, a) \rightarrow s'$ — what each action does
 - Goal test $\text{Goal?}(s)$ — is s a goal state?
 - Path cost $c(s, a, s')$ — cost of taking action a in s to reach s'
- A solution is a sequence of actions from s_0 to a goal state
- An optimal solution minimises total path cost

Example: Romania Road Map (AIMA §3)

Get from Arad to Bucharest — the canonical AI search example.

§2 Formulation

State Space

Cities in Romania (e.g., 20 cities)

Initial State

Arad

Actions

Drive to adjacent city

Transition Model

Map of road connections

Goal Test

Am I in Bucharest?

Path Cost

Sum of road distances (km)

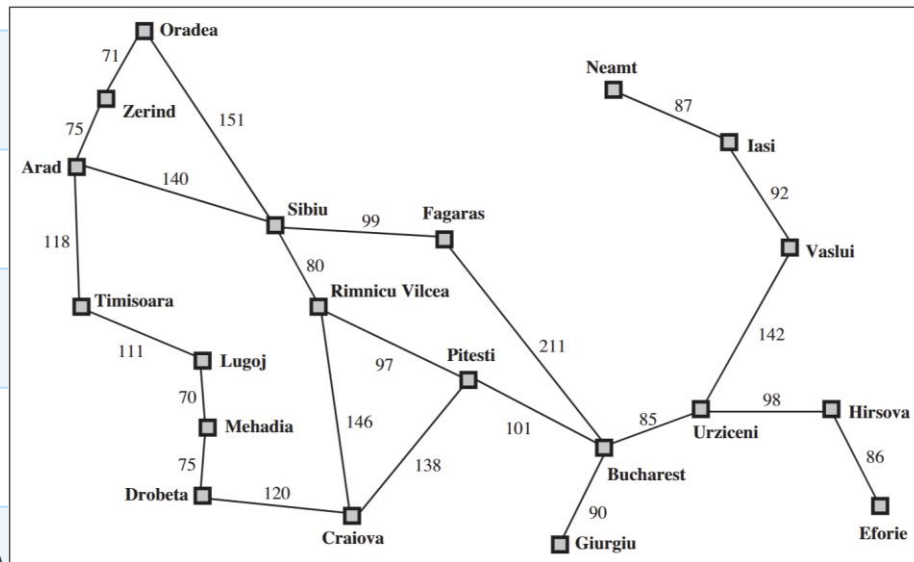


Figure 3.2 A simplified road map of part of Romania.

From this formulation, search algorithms explore the state space as a tree/graph.

Example: Solving the 8-puzzle

Get from Arad to Bucharest — the canonical AI search example.

State Space	Configuration of the 8 tiles											
Initial State	<table><tr><td>7</td><td>2</td><td>4</td></tr><tr><td>5</td><td></td><td>6</td></tr><tr><td>8</td><td>3</td><td>1</td></tr></table>			7	2	4	5		6	8	3	1
7	2	4										
5		6										
8	3	1										
Actions	Move a tile to the space: Left, Right, Up, or Down.											
Transition Model	Geometric											
Goal Test	Tiles arranged in sequence											
Path Cost	Number of moves											

	1	2
3	4	5
6	7	8

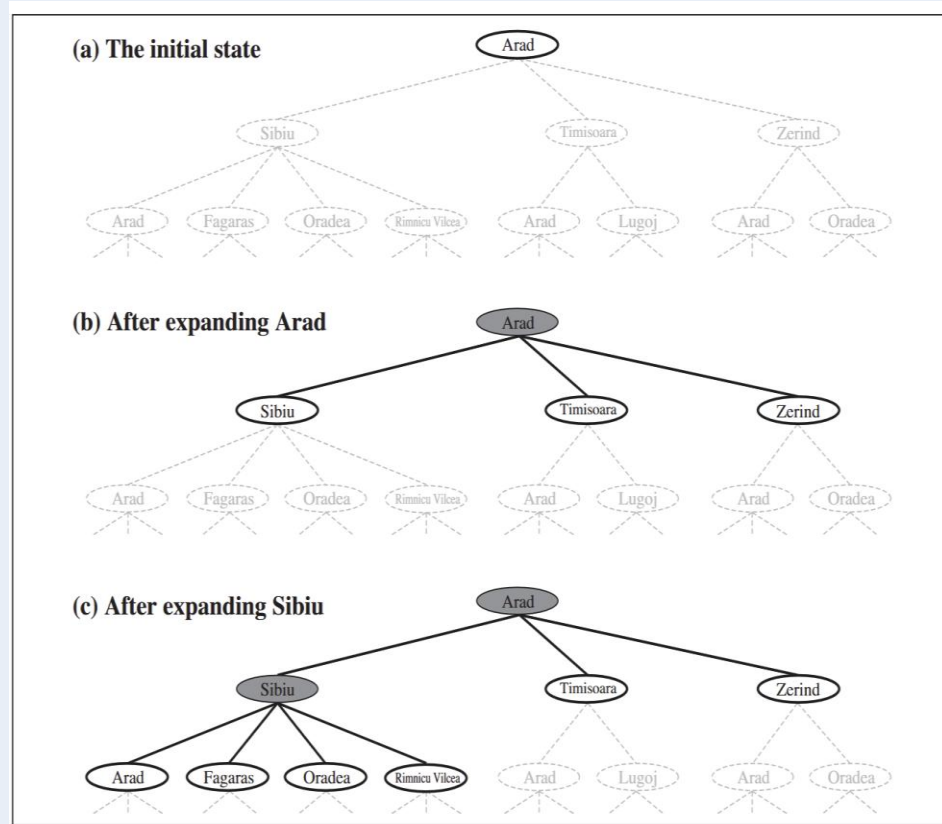
From this formulation, search algorithms explore the state space as a tree/graph.

Real-world problems

- A VLSI layout problem requires positioning millions of components and connections VLSI LAYOUT on a chip to minimize area, minimize circuit delays, minimize stray capacitances, and maximize manufacturing yield
- Robot navigation: continuous space of movements to achieve a particular goal
- Automatic assembly sequencing of complex objects by a robot.: find an order in which to assemble the parts of some object
- Protein design: find a sequence of amino acids that will fold into a three-dimensional protein with the right properties to cure some disease.

Search Trees and Graphs

- Search tree: generated by expanding nodes from root
- Node contains: state, parent, action, path cost $g(n)$
- Frontier (open list): nodes generated but not yet expanded
- Explored set (closed list): nodes already expanded
- Tree search: may revisit states (no closed list)
- Graph search: avoids revisiting states



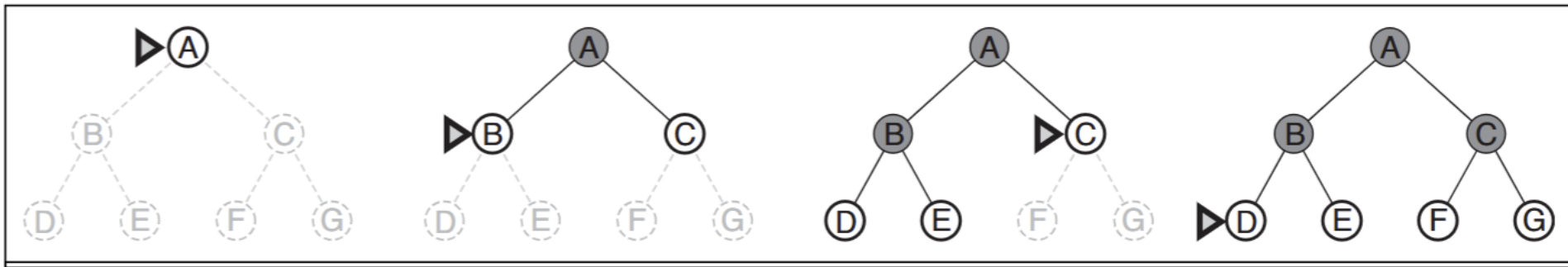
function GRAPH-SEARCH(*problem*) **returns** a solution, or failure
 initialize the frontier using the initial state of *problem*
 initialize the explored set to be empty
 loop do
 if the frontier is empty **then return** failure
 choose a leaf node and remove it from the frontier
 if the node contains a goal state **then return** the corresponding solution
 add the node to the explored set
 expand the chosen node, adding the resulting nodes to the frontier
 only if not in the frontier or explored set

Two types of search

- Uninformed or blind search: strategies have no additional information about states beyond that provided in the problem definition. All they can do is generate successors and distinguish a goal state from a non-goal state. All search strategies are distinguished by the order in which nodes are expanded.
- Informed Search or heuristic search: Strategies that know whether one non-goal state is “more promising”

Breadth-First Search (BFS)

- Strategy: expand shallowest unexpanded node (FIFO frontier)
- Complete: Yes — finds a solution if one exists (finite b)
- Optimal: Yes — if step costs are all equal (finds shortest path)
- Time complexity: $O(b^d)$ where b = branching factor, d = depth of shallowest goal
- Space complexity: $O(b^d)$ — keeps all nodes in memory — major limitation!
- Practical: BFS works well when d is small; memory becomes the bottleneck



$b=10, d=10$: $\sim 10^{10}$ nodes ≈ 10 GB. $b=10, d=12$: $\sim 10^{12}$ nodes ≈ 1 TB. Memory is the real enemy of BFS.

function BREADTH-FIRST-SEARCH(*problem*) **returns** a solution, or failure

node $\leftarrow$ a node with STATE = *problem*.INITIAL-STATE, PATH-COST = 0

if *problem*.GOAL-TEST(*node*.STATE) **then return** SOLUTION(*node*)

frontier $\leftarrow$ a FIFO queue with *node* as the only element

explored $\leftarrow$ an empty set

loop do

if EMPTY?(*frontier*) **then return** failure

node $\leftarrow$ POP(*frontier*) /* chooses the shallowest node in *frontier* */

 add *node*.STATE to *explored*

for each *action* **in** *problem*.ACTIONS(*node*.STATE) **do**

child $\leftarrow$ CHILD-NODE(*problem*, *node*, *action*)

if *child*.STATE is not in *explored* or *frontier* **then**

if *problem*.GOAL-TEST(*child*.STATE) **then return** SOLUTION(*child*)

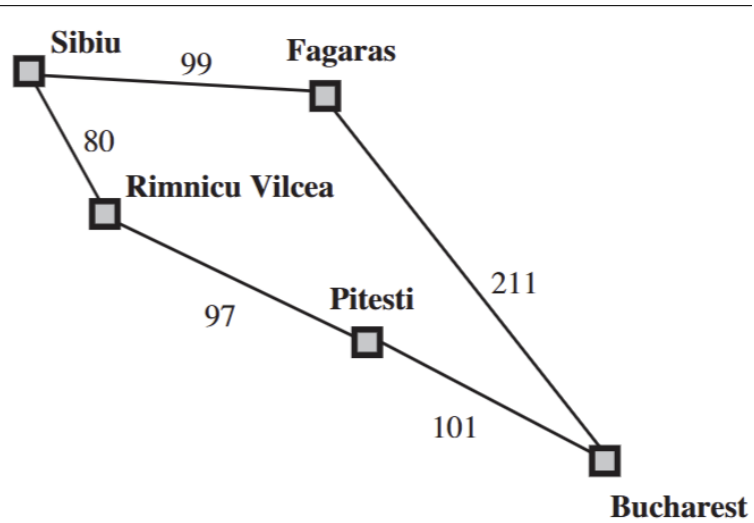
frontier $\leftarrow$ INSERT(*child*, *frontier*)

Uniform-Cost Search

Strategy: expand node with lowest current path-cost from the start. (use priority queue)

```
node ← a node with STATE = problem.INITIAL-STATE, PATH-COST = 0
frontier ← a priority queue ordered by PATH-COST, with node as the only element
explored ← an empty set
loop do
  if EMPTY?(frontier) then return failure
  node ← POP(frontier) /* chooses the lowest-cost node in frontier */
  if problem.GOAL-TEST(node.STATE) then return SOLUTION(node)
  add node.STATE to explored
  for each action in problem.ACTIONS(node.STATE) do
    child ← CHILD-NODE(problem, node, action)
    if child.STATE is not in explored or frontier then
      frontier ← INSERT(child, frontier)
    else if child.STATE is in frontier with higher PATH-COST then
      replace that frontier node with child
```

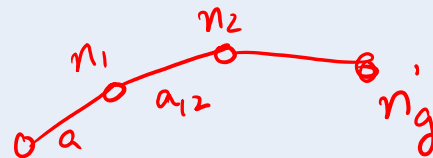
- Complete: Yes (if step costs $> \epsilon > 0$). Cycles of zero cost can cause infinite loop
- Optimal: Yes — always finds min-cost solution
- Time & Space: $O(b^{(1 + \lfloor C^*/\epsilon \rfloor)})$
- C^* = cost of optimal solution
- Dijkstra's algorithm is essentially UCS



Informed Search: Heuristic Functions

- A heuristic $h(n)$ estimates the cost from node n to the nearest goal
- $h(n) = 0$ if n is a goal state
- Informed search uses $h(n)$ to guide exploration toward the goal
- Examples:
 - Romania: $h_{sld}(n)$ = straight-line distance to Bucharest
 - 8-puzzle: h_1 = number of misplaced tiles
 - 8-puzzle: h_2 = sum of Manhattan distances
- Admissible heuristic: $h(n) \leq h^*(n)$ — never overestimates the true cost
- Consistent (monotone): $h(n) \leq c(n, a, n') + h(n')$ — triangle inequality
- Consistency implies admissibility;

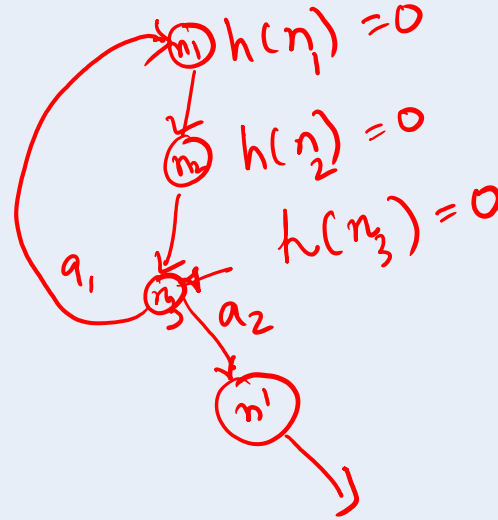
$$h(n) \leq c(n, a, n_1) + h(n_1) \leq c(n, a, n_1) + c(n_1, a_{12}, n_2) + h(n_2) = c(n \rightarrow n_g) n$$



Greedy Best-First

Greedy Best-First Search

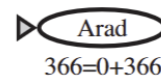
- Expand most promising node, best $h(n)$
- Complete: No (can loop)
- Optimal: No
- Fast in practice but may miss optimal path
- Like a greedy person reading a map



A* Search

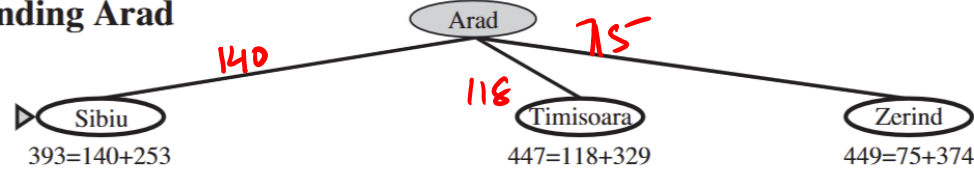
- Expand nodes in the order of $f(n) = g(n) + h(n)$
 - $g(n)$ = cost so far (from start to n)
 - $h(n)$ = estimated cost to goal
- Complete & Optimal with admissible h
- Optimal-efficient: no other algorithm expands fewer nodes (with same h)
- Space: still $O(b^d)$ — main drawback

(a) The initial state



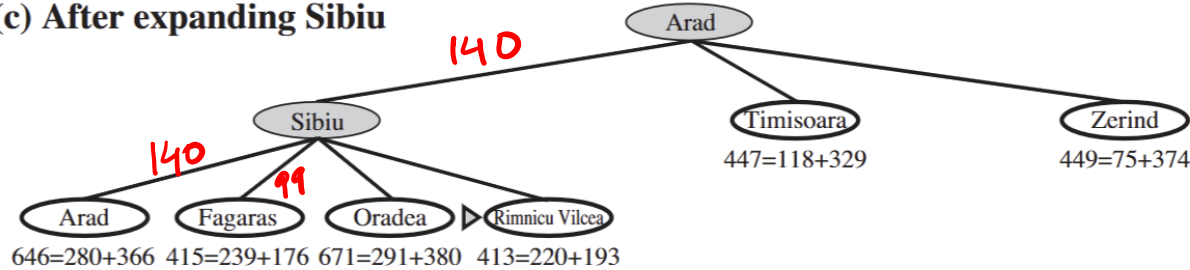
366

(b) After expanding Arad



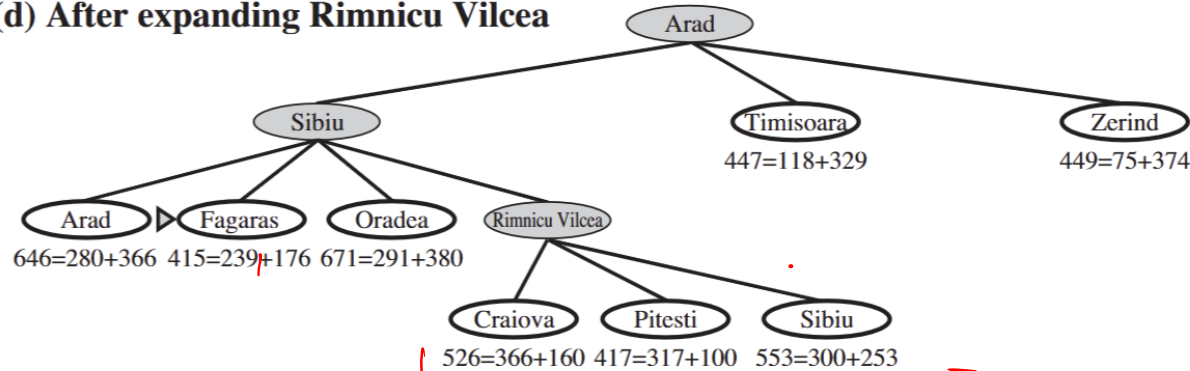
393

(c) After expanding Sibiu



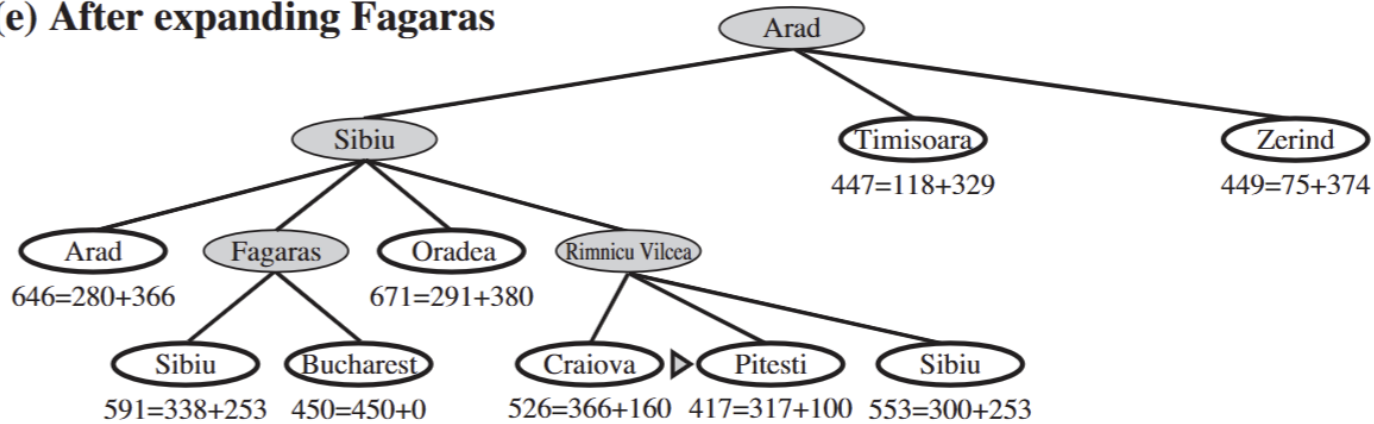
413

(d) After expanding Rimnicu Vilcea

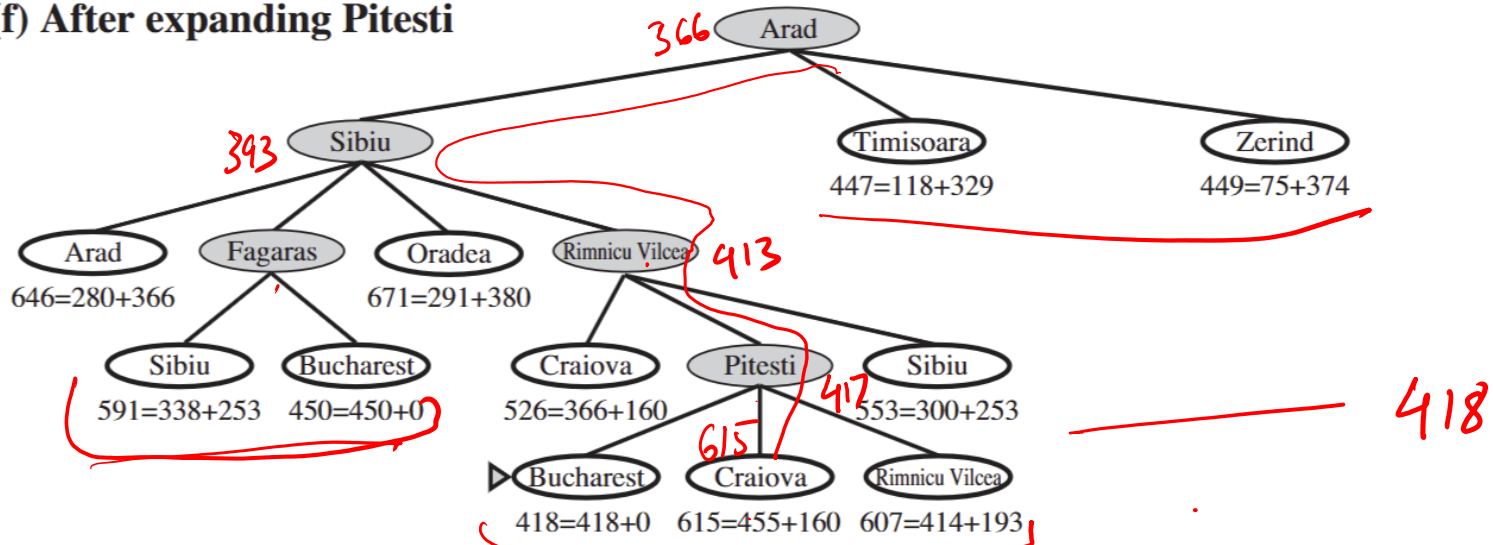


415

(e) After expanding Fagaras



(f) After expanding Pitesti



Optimality of A*

- If $h(n)$ is consistent, then the values of $f(n)$ along any path are non-decreasing.

$$f(n') = g(n') + h(n') = g(n) + c(n, a, n') + h(n') \geq g(n) + h(n) = f(n) .$$

- Whenever A* selects a node n for expansion, the optimal path to that node has been found.
 - Were this not the case, there would have to be another frontier node n' on the optimal path from the start node to n
 - f is nondecreasing along any path, n' would have lower f -cost than n and would have been selected first

Nice overview of A*: <https://www.redblobgames.com/pathfinding/a-star/introduction.html>

Beyond A*: Practical Variants

§4 Informed

- Memory-Bounded Heuristic Search
 - IDA* (Iterative-Deepening A*): uses f-cost threshold instead of depth — $O(bd)$ space
 - RBFS, SMA*: drop least-promising paths to free memory
- Heuristic Design
 - Relaxed problem: remove constraints $\rightarrow$ gives admissible heuristic automatically
 - Pattern databases: precomputed exact costs for subproblems
 - Dominant heuristic: h_2 dominates h_1 if $h_2(n) \geq h_1(n) \forall n \rightarrow$ use h_2 , fewer nodes expanded

Key Takeaways

01

Intelligent Agents

An agent perceives and acts. Rational agents maximise expected performance.

02

PEAS & Environments

Characterise any AI task: Performance, Environment, Actuators, Sensors.

03

Problem Formulation

State space, initial state, actions, transition model, goal test, path cost.

04

Uninformed Search

BFS, DFS, UCS, IDS — differ in completeness, optimality, time, space.

05

Informed Search

Heuristics $h(n)$ guide search. A^* is optimal with admissible heuristic.

06

Heuristic Quality

Better heuristics → fewer nodes expanded. Relaxation gives admissible h .

<https://aimacode.github.io/aima-exercises/search-exercises/>

References & Further Reading

Resources

Primary Textbook

- ▶ Russell, S. & Norvig, P. — Artificial Intelligence: A Modern Approach, 3rd edition
- ▶ Chapters 1–2 (Intelligent Agents), Chapter 3 (Search in Discrete Spaces).
- ▶ Free online version: aima.cs.berkeley.edu

Online Resources

- ▶ Norvig's AIMA Python code: github.com/aimacode/aima-python
- ▶ visualgo.net — animated visualisations of BFS, DFS

Other topics

- Adversarial search: relevant for playing games
- Planning-based search
- Handling uncertainty
- Probabilistic reasoning
 - Bayesian networks
- Reinforcement learning
- Agents in action: Today's LLM agents

My next course: CS 791

Advanced topics in probabilistic modeling that forms the foundations of modern AI.

Learning to represent, train, generate, and reason on object :

- High dimensional $x = \{x_1, \dots, x_n\}$, space of x is large
- Inter-dependent components

Examples: Image, Video, Time-series, Text